Each accessible event has an event handler, which is a block of code (typically a JavaScript function created by you as a programmer) that is executed when the event occurs. We call registering an event handler a block of code that is specified to run in response to an event. Remember that event handlers and event listeners are always used interchangeably for our purposes, though they do fit together, strictly speaking. The listener is the code that is executed when an event occurs, and the controller is the code that is executed in response to the event occurring.

Using event listeners

In web design, event listeners are one of the most commonly employed JavaScript structures. They allow us to add interactive features to HTML elements by "listening" to events on the web, such as when a user clicks a button, presses a key, or when an element loads.

We should do something after an event occurs.

Load, select, touchstart, mouseover, and **keydown** are the most popular events to "listen out for". MDN's Case Reference guide contains a list of all DOM events.

Below, we have listed three separate ways to make a JavaScript event listener:

- The global onevent attributes in HTML
- The event method in jQuery
- The addEventListener() method of the DOM API

How to Make a JavaScript Event Listener

We can listen to any of the events specified in MDN's Event Reference, including touch events, using native JavaScript. It's the most performance-friendly way to bring dynamic features to HTML elements since it doesn't need the usage of a third-party library.

The **addEventListener()** method, which is integrated into any modern browser, can be used to build an event listener in JavaScript.

Using simple JavaScript and the **addEventListener()** method, is how our alarm button example would look.

In native JavaScript, we must first pick the DOM variable on which the event listener would be applied. The **querySelector()** method finds the first variable that fits a selector. As a result, it selects the first **<button>** element on the page in our case.

When the user presses the switch, the custom **alertButton()** feature is named as a callback function.